Next.js App Router: When to Use Link vs Router

This guide explains when to use <Link> and when to use router.push() in Next.js App Router, including examples with query parameters and dynamic routes.

1) Use <Link> for Static Navigation

<Link> is the preferred way to navigate between routes in Next.js.
It provides automatic prefetching, client-side navigation, and performance benefits.

Example (with query params):

```
<Link href="/CoffeeProject/caffeePage1?type=espresso">
  Espresso
</Link>
```

Then read it using:

```
const searchParams = useSearchParams();
const type = searchParams.get("type");
```

2) Use router.push() for Dynamic or Conditional Navigation

Use router.push() when you need to perform logic before navigating.

Example:

```
const router = useRouter();
const handleClick = () => {
  saveData();
  router.push("/CoffeeProject/caffeePage1?type=latte");
};
```

3) Dynamic Route Example

If you prefer cleaner URLs like /CoffeeProject/caffeePage1/espresso,

create a dynamic route folder [type]/page.tsx.

Access params like this:

```
export default function CoffeeTypePage({ params }) {
  return <p>You selected: {params.type}</p>;
}
```

Summary Table

Case: Static navigation between pages

-> Use: <Link>

-> Why: Prefetching, declarative

Case: Conditional navigation (after logic)

-> Use: router.push()

-> Why: Programmatic control

Case: Pass data in URL

-> Use: <Link href="/page?key=value" />

-> Why: Simple and SEO-safe

Case: Dynamic segments

-> Use: /[param]/page.tsx

-> Why: Clean and structured URLs

Recommendation

Use <Link> whenever possible for navigation.

Use router.push() only when user actions or async logic determine where to go.